Karlstad university
Department of mathematics and computer science
DVGA28 Computer systems I

14th February 2026
Kerstin Andersson

Exercises

Module 2—Computer engineering, 6 ECTS credits

1 Data representation and arithmetic

1. Perform the subtraction $12 - 45$ using MARS and by using addi and sub instructions (only). What is the result? Why?

2. Perform the following calculations on a processor with word length 1 byte:

00101101	11111111	00000000	11110111
+ 01101111	+ 11111111	- 11111111	- 11110111

   What is the result? Right or wrong? Interpret the binary sequences as both positive numbers and numbers in two's complement.

3. Perform the multiplication $35 \cdot 8$ using MARS and by using addi and sll instructions (only). Perform the integer division $35/4$ by using addi and srl instructions (only).

4. Multiply the two 8-bit numbers `0x33` and `0x55` using only addition, suubtraction and shift instructions.

5. Perform the calculation $5 \cdot (-6)$ in two's complement on a 5-bit machine. Interpret the result using 5 and 10 bits, respectively.

6. Division is performed as a set of shift operations and subtractions. In the example below the integer division using nominator 74 (1001010) and denominator 8 (1000) is performed:

   ```
         1001      quotient
   1000|1001010    denominator, nominator
       -1000
       ------
          10
          101
          1010
         -1000
         -----
           10      remainder
   ```

   Repeat the procedure (in binary) using nominator 74 and denominator 21.

7. Interpret the following binary string in multiple ways: `10001111111011111100000000000000`.

8. Convert the following decimal numbers to IEEE floating-point numbers in single precision (in hexadecimal form): 19, 5/32, −5/32, 6.125, 27.211

9. Convert the following hexadecimal numbers (IEEE floating-point numbers in single precision) to decimal numbers: `0x42e48000`, `0x3f880000`, `0x00800000`, `0xc7f00000` and `0x40ca8f5c`

10. Convert the decimal numbers 19 and 6.125 to IEEE floating-point numbers in single precision and then add the numbers (adjust exponents and then add the significands). Finally convert the sum to a decimal number.

11. Write the following program in a file:

```
        .data
var1:   .float 3.14
var2:   .float 7
sum:    .float 0

        .text
        lwc1    $f1, var1
        lwc1    $f2, var2
        add.s   $f0, $f1, $f2
        swc1    $f0, sum
```

Assemble and run the program (single step) in MARS. See what happens in memory (in hexadecimal) and registers (Coproc 1). Convert the two numbers in `var1` and `var2` (manually) to IEEE floating-point numbers, add the IEEE numbers and compare to the three hexadecimal values in `var1`, `var2` and `sum`. Use other numbers as well.

Solutions

1 Data representation and arithmetic

1. 12 = `0000 1100` = `0x0c` and 45 = `0010 1101` = `0x2d`. In two's complement −45 is `1101 0010` + `1` = `1101 0011` = `0xd3`. The sum of 12 and (−45) is `1101 1110` = `0xdf`. Sign extension gives `0xffffffdf`.

2. The subtractions are replaced by additions by negating the second operand: -11111111 = 00000001 and -11110111 = 00001001. The results from the calculations:

00101101	11111111	00000000	11110111
+ 01101111	+ 11111111	+ 00000001	+ 00001001
10011100	11111110	00000001	00000000

   Interpretation:

   - Positive number: 45 + 111 = 156 (right). Two's complement: 45 + 111 = -100 (wrong).
   - Positive number: 255 + 255 = 254 (wrong). Two's complement: -1 + (-1) = -2 (right).
   - Positive number: 0 - 255 = 1 (wrong). Two's complement: 0 - (-1) = 1 (right).
   - Positive number: 247 - 247 = 0 (right). Two's complement: -9 - (-9) = 0 (right).

3. The two instructions "addi $t0, $zero, 35" and "sll $t0, $t0, 3" results in `0x118` = 280 = 35 · 8. The two instructions "addi $t0, $zero, 35" and "srl $t0, $t0, 2" results in `0x8` = 8 = 35/4 (integer division).

4. `0x33` = `00110011` = $2^5 + 2^4 + 2^1 + 2^0$. Left shift `0x55` 5, 4 and 1 steps, respectively, gives: `0xaa0`, `0x550` and `0xaa`. Adding the numbers gives: `0xaa0` + `0x550` = `0xff0`, `0xff0` + `0xaa` = `0x109a` and `0x109a` + `0x55` = `0x10ef`. In total three shift operations and three additions, i.e., faster than a multiplication.

5. The number range for a 5-bit machine is: $-2^4 \leq number \leq 2^4 - 1$, i.e., $-16 \leq number \leq 15$. In two's complement 5 = `00101` and −6 = `11010`. The multiplikationen is given by:

00101	00101	00101
* 11010	* 11010	* 11010
00000	00000	0000000000
+ 00101	+ 00101	+0000001010
+ 00000	+ 00000	+0000000000
+ 00101	+ 00101	+0000101000
- 00101	+11011	+1110110000
		1111100010

   Using 5 bits the result is `00010` = 2, which is wrong (not enough bits, the expected result (−30) is outside the number range). Using 10 bits the result is `1111100010` = −30, since `0000011101` + 1 = `0000011110` = 30. Right answer.

6. The calculation is given by:

```
      11        quotient
10101|1001010   denominator, nominator
     - 10101
      100000
     - 10101
        1011    remainder
```

7.
   - Positive integer: $(2^{17} + 2^{13} + 2^{12} + 2^{11} + 2^{10} + 2^9 + 2^8 + 2^7 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 1) \cdot 2^{14} = 147391 \cdot 16384 = 2414854144$.
   - Negative integer in two's complement: The corresponding positive number is obtained (after inverting and addition by 1) as `0111 0000 0001 0000 0100 0000 0000 0000` $= 2^{30} + 2^{29} + 2^{28} + 2^{20} + 2^{14} = 1073741824 + 536870912 + 268435456 + 1048576 + 16384 = 1880113152$. The negative number is -1880113152.
   - MIPS instruction: opcode = `100011` means `lw`. The following two 5-bit fields are registers (`11111` = 31 and `01111` = 15). The final 16 bits (`0xc000`) is the constant. The instruction is `lw` \$15, `0xc000`(\$31).
   - Floating-point number in single precision: The sign bit is 1 (negative number). The exponent field `0001 1111` givs the exponent $31 - 127 = -96$. The significand is `1.110111111` $= 1 + 2^{-1} + 2^{-2} + 2^{-4} + 2^{-5} + 2^{-6} + 2^{-7} + 2^{-8} + 2^{-9} = 1.873046875$. Total: -1.873046875 $\cdot 2^{-96}$ = -2.36412 $\cdot 10^{-29}$.

8.
   - $19 = 10011 = +1.0011 \cdot 2^4$. Sign bit: 0. Exponent field: 127+4 = 128 + 2 + 1 = 10000011. Significand field: 001 1000 0000 0000 0000 0000. Total: 0100 0001 1001 1000 0000 0000 0000 0000 = 0x41980000.
   - $5/32 = 5 \cdot 2^{-5} = +101 \cdot 2^{-5} = +1.01 \cdot 2^{-3}$. Exponent field: 127-3 = 124 = 64 + 32 + 16 + 8 + 4 = 01111100. Significand field: 010 0000 0000 0000 0000 0000. Total: 0011 1110 0010 0000 0000 0000 0000 0000 = 0x3e200000.
   - As in the previous exercise, but sign bit = 1. Total: 1011 1110 0010 0000 0000 0000 0000 0000 = 0xbe200000.
   - $6.125 = (1 + x) \cdot 2^y$, $0 \leq x < 1$, y is an integer, which makes $y = 2$ and $x = 0.53125$. By repeated multiplications by 2 you arrive at 0.53125 = 0.100 0100 0000 0000 0000 0000. Sign bit is 0 and exponent field is 127 + 2 = 129 = 1000 0001. Total: 0100 0000 1100 0100 0000 0000 0000 0000 = 0x40c40000.
   - $27.211 = (1 + x) \cdot 2^y$, $0 \leq x < 1$, y is an integer, which makes $y = 4$ and $x = 0.7006875$. By repeated multiplications by 2 you arrive at 0.7006875 = 0.101 1001 1011 0000 0010 0000. Sign bit is 0 och exponent field is 127 + 4 = 131 = 1000 0011. Total: 0100 0001 1101 1001 1011 0000 0010 0000 = 0x41d9b020.

9.
   - 0x42e48000 = 0 10000101 110 0100 1000 0000 0000 0000. Sign bit is 0 (positive number). Exponent field is 1000 0101 = 128 + 5. Subtract 127 for obtaining the real exponent (6). The number is: + $1.11001001 \cdot 2^6 = (1 + 0.5 + 0.25 + 2^{-5} + 2^{-8}) \cdot 2^6 = 64 + 32 + 16 + 2 + 0.25 = 114.25$.
   - 0x3f880000 = 0 01111111 000 1000 0000 0000 0000 0000. Sign bit is 0 (positive number). Exponent field is 01111111 = 127. Subtract 127 for obtaining the real exponent (0). The number is: + $1.0001 \cdot 2^0 = 1 + 0.0625 = 1.0625$.
   - 0x00800000 = 0 00000001 000 0000 0000 0000 0000 0000. Sign bit is 0 (positive number). Exponent field is 00000001 = 1. Subtract 127 for obtaining the real exponent (-126). The number is: + $1.0 \cdot 2^{-126} = 1.1754944 \cdot 10^{-38}$.
   - 0xc7f00000 = 1 10001111 111 0000 0000 0000 0000 0000. Sign bit is 1 (negative number). Exponent field is 1000 1111 = 128 + 15. Subtract 127 for obtaining the real exponent (16). The number is: - $1.111 \cdot 2^{16} = -15 \cdot 2^{13} = -122880$.
   - 0x40ca8f5c = 0 10000001 100 1010 1000 1111 0101 1100. Sign bit is 0 (positive number). Exponent field is 1000 0001 = 128 + 1. Subtract 127 for obtaining the real exponent (2). The number is: + $1.10010101000111101011 \cdot 2^2 = (1 + 0.5 + 0.0625 + 2^{-6} + 2^{-8} + 2^{-12} + 2^{-13} + 2^{-14} + 2^{-15} + 2^{-17} + 2^{-19} + 2^{-20} + 2^{-21}) \cdot 2^2 = 6.33$.

10. $19 = +1.0011 \cdot 2^4$ and $6.125 = +1.100\ 01 \cdot 2^2$. Before adding the significands the smallest number is right shifted (two steps) to adjust the exponent.

$$\begin{array}{r} 1.0011000 \\ +\ 0.0110001 \\ \hline 1.1001001 \end{array}$$

Total: 0100 0001 1100 1001 0000 0000 0000 0000 = 0x41c90000.

11. $7 = 111 = 1.11 \cdot 2^2$: sign bit = 1, exponent field = 127 + 2 = 129 = 1000 0001, significand field (except '1.'): 1100 0000 ... 000. Total: 0100 0000 1110 0000 ... 0000 = 0x40e00000.

    $3.14 = (1 + x) \cdot 2^y$, $0 \leq x < 1$, y is an integer, which makes $y = 1$ and $x = 0.57$. By repeated multiplications by 2 you arrive at 0.57 = 0.1001 0001 1110 1011 1000 010. Sign bit is 0 och exponent field is 127 + 1 = 128 = 1000 0000. Total: 0100 0000 0100 1000 1111 0101 1100 0010 = 0x4048f5c2.

    Before adding the significands the smallest number is right shifted (one step) to adjust the exponent: 1.1100 0000 ... 000 + 0.1100 1000 1111 0101 1100 001 = 10.1000 1000 1111 0101 1100 001. Right shift (one step) results in: significand filed = 0100 0100 0111 1010 1110 000, exponent field = 127 + 3 = 130 = 1000 0010, sign bit = 0. Total: 0100 0001 0010 0010 0011 1101 0111 0000 = 0x41223d70

    In memory at addresses `var1`, `var2` and `sum` you will find the numbers 0x4048f5c3, 0x40e00000 and 0x41223d71, respectively. The last digit may differ with what you obtaind manually (rounding error).